



IIC2343 - Arquitectura de Computadores (II/2026)

Ayudantía 8

Ayudantes: Ignacio Gajardo (gajardo.ignacio@uc.cl), Mario Rojas (mario.denzel@estudiante.uc.cl),
Nicolás Romo Aubele (nroma@uc.cl), Carlos Canales (carlos.canales@estudiante.uc.cl)

Pregunta 1: DCCálculo - RISC-V al rescate

Te encuentras estudiando para un ramo de la Universidad, cuando en un ejercicio de ayudantía te aparece la temible función **factorial**: $x!$. Tu compañero te entrega la siguiente propuesta en RISC-V para calcularla:

```
.data
x: .word 8

.text
lw t0, x           # Contador de cuántas multiplicaciones nos faltan
addi s0, zero, 1   # Constante = 1
addi sp, sp, -4     # Respalda ra
sw ra, 0(sp)
jal ra, factorial_x # Llamada a subrutina
lw ra, 0(sp)

# ecall para imprimir el resultado (código 1)
addi a7, zero, 1    # a7 -> Código de la operación a realizar
add a0, zero, t0     # a0 -> (Si es necesario) Argumentos de la operación a realizar
ecall

# ecall para terminar el programa (código 10)
addi a7, zero, 10
ecall

factorial_x:
    beq t0, s0, end_factorial_x
    addi sp, sp, -8
    sw ra, 0(sp)      # Respalda de ra
    sw t0, 4(sp)      # Guardar qué números tengo que multiplicar
    addi t0, t0, -1    # Decrezco el contador
    jal ra, factorial_x # Vuelvo a llamar a la subrutina
    lw ra, 0(sp)      # Recuperamos ra

    # Multiplicación
    lw t1, 4(sp)      # Recuperamos el número que toca multiplicar ahora
    addi sp, sp, 8     # Reestablecemos el SP
    mul t0, t0, t1     # t0 = t0 * t1

end_factorial_x:
    jalr zero, 0(ra)  # Equivalente a RET, sale de la función
```

A partir del código presentado y considerando la **convención de llamadas de RISC-V** vista en clases:

- a) ¿Cumple este código con la convención de llamadas de RISC-V?
- b) Justifique detalladamente su respuesta indicando el uso de registros y el manejo de contexto.

Pregunta 2: Análisis de Código: La Función Misteriosa

El siguiente código recursivo fue escrito en **Assembly RISC-V**, pero los nombres de las funciones y variables que participan en la subrutina han sido ofuscados.

```
.data
a: .word 2
b: .word 5
.text
main:
    la t0, a
    lw a0, 0(t0)
    la t0, b
    lw a1, 0(t0)
    call fun_mistery
    mv s0, a0
    j fin
fun_mistery:
    addi sp, sp, -8
    sw ra, 4(sp)
    sw a0, 0(sp)
    blez a1, caso_a
    addi a1, a1, -1
    call fun_mistery
    lw t1, 0(sp)
    mul a0, a0, t1
    j retorno
caso_a:
    li a0, 1
retorno:
    lw ra, 4(sp)
    addi sp, sp, 8
    ret
fin:
    li a7, 1
    ecall
    li a7, 10
    ecall
```

Responda a las siguientes preguntas basándose exclusivamente en el análisis del código:

- ¿Qué hace la subrutina `fun_mistery` en función de las variables `a` y `b`?
- Al ejecutar el bloque `fin`, ¿Cuál será el valor numérico que se imprimirá en la consola?
- En el prólogo de `fun_mistery`, se observa que el registro `a0` es respaldado en el *stack*, pero `a1` no lo es. Justifique esta decisión basándose en la **Convención de llamadas** y el propósito general del algoritmo.

Pregunta 3: Corrección de código en RISC-V

El siguiente programa, desarrollado en el Assembly del computador básico visto en clases, **debe-se verificar si el número n es múltiplo de m** :

```
.data
    x_value: .word 7
    n:       .word 3
    x_pow_n: .word 0
.text
main:
    lw a0, x_value
    lw a1, n
    la s0, x_pow_n
    addi s1, zero, 1
    jal ra, pow
    sw a0, 0(s0)
    addi a7, zero, 10
    ecall

pow:
    beq a1, s1, end
    addi sp, sp, -4
    sw a0, 0(sp)
    addi a1, a1, -1
    jal ra, pow
    lw t0, 0(sp)
    addi sp, sp, 4
    mul a0, a0, t0

end:
    jr ra
```

Sin embargo, no lo hace correctamente. Si compila y ejecuta el código, se dará cuenta que no entrega el resultado esperado. **Busque** el error y, una vez encontrado, **arréglelo** para que el fragmento anterior entregue el resultado esperado.

Feedback ayudantía

Escanee el QR para entregar feedback sobre la ayudantía.

